

RAG and Large Language Models - Practice Document

This short reference document is designed for practicing a Retrieval-Augmented Generation (RAG) pipeline. It contains clear facts that can be extracted, chunked, embedded, retrieved, and used to answer questions.

1. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, commonly called RAG, is a technique that combines information retrieval with a large language model. Instead of relying only on the model's internal knowledge, a RAG system first searches an external knowledge source for relevant information. The retrieved information is then added to the model's prompt as context before the model generates an answer.

A basic RAG pipeline usually contains four main stages: document preparation, retrieval, prompt construction, and generation. Document preparation may include extracting text, splitting it into chunks, and converting those chunks into vector embeddings.

2. Chunking

Chunking means splitting a long document into smaller pieces before indexing it. A chunk should be large enough to preserve useful meaning but small enough to retrieve precise information. If chunks are too large, retrieval may return a lot of irrelevant text. If chunks are too small, important context may be separated.

Chunk overlap is sometimes used so that information near the boundary of one chunk also appears in the next chunk. For example, a system might use chunks of 500 characters with an overlap of 100 characters.

3. Embeddings and Vector Search

An embedding is a numerical vector that represents the semantic meaning of text. Texts with similar meanings usually have vectors that are close to each other in the embedding space.

During indexing, each document chunk is converted into an embedding and stored. When a user asks a question, the question is also converted into an embedding. The system then compares the query vector with the stored chunk vectors to find the most similar chunks.

Cosine similarity is a common metric for comparing embeddings. A higher cosine similarity usually indicates that two vectors point in more similar directions and may represent more semantically related text.

4. Retrieval

The retrieval stage selects the most relevant chunks for the user's question. A system may retrieve the top three, top five, or another number of chunks depending on the application. Retrieving too many chunks can increase cost and introduce irrelevant context, while retrieving too few can miss important evidence.

Retrieval quality is critical because the language model can only use the evidence that is provided to it. If the retriever selects the wrong chunks, the final answer may be incomplete or incorrect even when the language model is strong.

5. Prompt Construction and Generation

After retrieval, the selected chunks are inserted into a prompt together with the user's question and instructions. A useful instruction may tell the model to answer only from the provided context and to say when the context does not contain enough information.

The language model then performs inference and generates an answer. In a RAG system, the model itself does not normally search the vector database. The application performs retrieval first and then supplies the retrieved text to the model.

6. Why RAG is Useful

RAG can help reduce hallucination by grounding model responses in external evidence. It also allows an application to use private or frequently updated information without retraining the entire language model.

Common RAG use cases include question answering over company documents, legal documents, technical manuals, policies, research papers, and internal knowledge bases.

7. Limitations and Evaluation

RAG does not guarantee a correct answer. Problems can occur during text extraction, chunking, embedding, retrieval, prompt construction, or generation. For this reason, RAG systems should be evaluated as pipelines rather than evaluating only the final language model.

Useful evaluation dimensions include retrieval relevance, answer correctness, faithfulness to the retrieved context, latency, and cost. A grounded answer should be supported by the retrieved evidence and should not invent unsupported facts.

Practice questions

1. What are the four main stages of a basic RAG pipeline?
2. Why is chunking necessary?
3. What is an embedding?
4. What does cosine similarity measure?
5. Why can retrieving too many chunks be a problem?
6. Does the LLM normally search the vector database directly?
7. How can RAG reduce hallucination?
8. What should be evaluated in a RAG system?